AI-Native Grocery Merchant

Document 02 — Functional Requirements Specification

Status	LOCKED — Development Source of Truth
Depends On	Document 01 — Development Master Specification
Purpose	Define exact functional behavior of the MVP
Scope	Development only; no pitch or marketing requirements

Functional contract: Every feature must have defined inputs, processing rules, outputs, failure behavior and acceptance criteria.

1. Functional Scope

This document translates the locked product definition into implementable functional requirements. The requirements below are mandatory for the MVP unless explicitly marked as supporting behavior.

The canonical transaction is: user goal → requirement extraction → catalog/research → optimization → incentives → two baskets → AI recommendation → user choice → policy authorization → Razorpay payment → verification → audit.

2. Actors and Responsibilities

Actor	Responsibility
User	Provides goal, budget and mandate; reviews AI recommendation; chooses one basket; completes payment.
AI Agent	Understands intent; researches; optimizes; evaluates incentives; generates baskets; recommends one; explains decisions; recovers from
Commerce Backend	Owns catalog, stock, prices, cart, totals and order state.
Policy Engine	Independently determines whether a selected financial action is authorized.
Razorpay Test Mode	Creates and processes the payment transaction in the test environment.
Audit System	Records important proposals, decisions, authorization results and payment state transitions.

3. FR-01 — User Shopping Goal

Input: Natural-language grocery request and budget. Optional quality preference or constraints may be supplied.

Processing: The system accepts the request and passes it to the AI planning workflow. The budget is interpreted as a hard maximum unless the mandate imposes a lower limit.

Output: A normalized shopping intent containing purpose, servings/context, budget and extracted constraints.

Failure: If the request is too ambiguous to produce a safe plan, the AI must ask for the minimum required clarification rather than inventing requirements.

Acceptance: A request such as “pasta for 4, budget ■1,000” creates a structured planning input without requiring the user to provide a SKU list.

4. FR-02 — Requirement Extraction

Input: Normalized shopping intent.

Processing: AI determines the ingredients/product categories and approximate quantities required to satisfy the stated goal.

Output: Structured requirements such as ingredient/category, target quantity, acceptable units and constraints.

Rules: Requirements must be sufficient for catalog search but must not create unnecessary products. Approximate culinary quantities are acceptable for the MVP when the goal does not specify exact quantities.

Acceptance: The AI can convert a meal goal into a usable candidate list before product selection.

5. FR-03 — Catalog Search

Input: Structured requirements and search terms.

Processing: Backend returns matching merchant products with authoritative SKU identity, price, unit, pack size, stock and available metadata.

Output: Candidate product set for AI comparison.

Rules: AI may rank candidates, but backend catalog values remain authoritative. AI cannot invent a product, price or stock state.

Acceptance: Search returns reproducible candidates from controlled merchant data.

6. FR-04 — Product Quality / Review Research

Input: Candidate products and available evidence.

Processing: AI summarizes relevant ratings, comments/review signals and merchant-provided quality information available to the system.

Output: Quality assessment with evidence references/signals and a decision-oriented conclusion.

Rules: Evidence must be distinguished from inference. The AI must not present unverified allegations as facts. The MVP uses controlled/seeded evidence rather than an unrestricted web crawler.

Acceptance: The agent can reject a cheaper product when available evidence indicates unacceptable quality/reliability.

7. FR-05 — Quantity and Pack Optimization

Input: Required quantity plus candidate pack sizes.

Processing: AI/system evaluates combinations of packs to satisfy the requirement while minimizing waste/cost subject to quality constraints.

Output: Selected pack combination and calculation of delivered quantity and cost.

Rules: Extra quantity must have a reasonable justification. The system must not buy unnecessary excess solely to obtain a discount.

Acceptance: For six eggs, the system can select three 2-packs at ■36 over a poor-quality 6-pack at ■36 when evidence supports the better-quality option.

8. FR-06 — Deal Optimization

Input: Candidate products, quantities, packs, prices, quality evidence and eligible incentives.

Processing: Evaluate total economic value rather than selecting the lowest unit price blindly.

Output: Candidate baskets scored against cost, quality, quantity fit and incentives.

Rules: Budget is a ceiling; unused budget is not a problem. The agent must not add unnecessary products simply to spend more.

Acceptance: The system can produce different but valid baskets for cost-first and quality-first objectives.

9. FR-07 — Smart Voucher Decision

Input: Basket, voucher rules, current saving, eligibility, expiry and known future-value signals.

Processing: Determine whether the voucher should be used now or preserved.

Output: USE or SAVE decision plus explanation and calculated current saving.

Rules: Voucher presence does not imply use. A 5% voucher saving ■5 on a ■100 snack may be saved. The AI must not add unnecessary items to unlock a voucher.

Acceptance: A materially valuable voucher can be used; a low-value voucher with future utility can be saved; the decision is visible to the user.

10. FR-08 — Loyalty Reward Decision

Input: Basket, loyalty state and reward rules.

Processing: Check eligibility and evaluate whether consuming the reward provides meaningful value for the current purchase.

Output: Use/save decision, applied value if used, and explanation.

Rules: Rewards cannot be assumed eligible. The server verifies reward conditions before application.

Acceptance: The AI can incorporate an eligible loyalty benefit without changing authoritative price or reward state itself.

11. FR-09 — Best Value Basket

Objective: Lowest practical cost while satisfying requirements and maintaining acceptable quality/reliability.

Input: Candidate products, pack combinations and incentive options.

Output: Complete basket, gross total, discounts, final payable, quality summary and explanation.

Acceptance: Basket is valid, fulfills requirements, contains no unjustified items, and remains within the mandate.

12. FR-10 — Best Quality Basket

Objective: Highest practical quality/reliability achievable within the user's authorized budget.

Input: Same candidate universe and evidence used for optimization.

Output: Complete basket, gross total, discounts, final payable, quality summary and explanation.

Rules: It must remain within the budget. If the ideal quality combination exceeds the budget, the system must find the best feasible alternative.

Acceptance: The basket improves quality/reliability relative to Best Value where meaningful, without violating the hard budget.

13. FR-11 — AI Recommendation

Input: Best Value and Best Quality baskets plus user's stated constraints/preferences.

Processing: AI recommends one option and explains the trade-off. It must not claim that the recommendation is the only valid choice.

Output: Recommended basket, rationale and alternative basket.

Acceptance: User can understand why the AI recommended one option and can choose the other.

14. FR-12 — User Basket Selection

Input: User selection of Best Value or Best Quality.

Processing: Backend records the selected option and reconstructs/revalidates its authoritative basket.

Output: Selected basket ready for final policy validation.

Rules: User selection is required before payment in the MVP. Selection does not itself authorize payment.

Acceptance: The user can override the AI recommendation and choose the other valid basket.

15. FR-13 — Mandate Management

Input: User-defined authorization constraints.

Required fields: agent ID, mandate ID, maximum spend, allowed categories, maximum per-item amount, purpose and validity.

Processing: Server stores and enforces mandate. AI can read mandate context but cannot modify it during shopping.

Acceptance: A selected basket outside the mandate cannot proceed to payment.

16. FR-14 — Deterministic Policy Engine

Input: Selected final basket, authoritative catalog state, incentive state and mandate.

Evaluation order: MANDATE_VALID → CATEGORY_ALLOWED → STOCK_AVAILABLE → MAX_PER_ITEM → INCENTIVE_VALIDITY/ELIGIBILITY → FINAL_AMOUNT_CALCULATION → MAX_SPEND.

Output: Structured ALLOW or DENY result, rule identifier and reason.

Rules: The LLM cannot override a denial. Final payable is server-calculated. Policy must be re-run immediately before money action.

Acceptance: A ■850 final basket under an ■800 mandate is denied with MAX_SPEND.

17. FR-15 — Cart Management

Input: Authorized basket mutations.

Processing: Server adds/removes items only after relevant validation; server recalculates totals.

Output: Current cart state, quantities, gross amount, discounts and final payable.

Rules: Client-provided totals are informational only. Every financial mutation must be auditable.

Acceptance: Cart total always derives from authoritative product and incentive state.

18. FR-16 — Checkout Preparation

Input: Selected basket and current server state.

Processing: Re-fetch/revalidate product price, stock, incentive eligibility, mandate and final total; run policy engine.

Output: Authorized payment amount and server-generated checkout/order request.

Acceptance: No Razorpay order is created if final policy result is DENY.

19. FR-17 — Razorpay Test Mode Payment

Input: Authorized server-calculated final amount.

Processing: Backend creates a Razorpay Test Mode order and the frontend opens the supported checkout flow.

Output: Razorpay order/payment identifiers and transaction state.

Rules: The amount must not come directly from the LLM or browser. No real-money payment is permitted in the MVP.

Acceptance: A successful Test Mode payment creates a traceable order and payment state.

20. FR-18 — Payment Verification & Webhooks

Input: Razorpay payment callback/webhook events.

Processing: Backend verifies payment state and validates webhook authenticity according to the integration design; duplicate events are handled idempotently.

Output: Authoritative order/payment status.

Acceptance: Client-side success alone cannot mark an order as paid.

21. FR-19 — Audit Trail

Input: Meaningful AI, policy, cart, incentive and payment events.

Processing: Record event type, timestamp, actor/context, relevant identifiers, decision, reason and financial values where applicable.

Output: Queryable audit history for the transaction/agent.

Acceptance: A judge/developer can trace a transaction from AI proposal through authorization to payment verification.

22. FR-20 — Graceful Failure and Recovery

Trigger: Policy denial or changed merchant state makes the proposed basket invalid.

Processing: Backend returns structured denial reason. AI uses the reason to re-optimize without changing the mandate.

Output: Compliant alternative basket or a clear request for user action if recovery is impossible.

Acceptance: For an ■850 proposal under an ■800 limit, the agent can produce a compliant alternative such as ■774 and explain the change.

23. Cross-Feature Business Rules

- The user's budget is a hard ceiling, not a target.
- Final payable amount is authoritative only when calculated by the backend.
- Gross basket value, discount value and final payable must remain separately represented.
- Quality optimization must not knowingly select an unacceptable product merely because it is cheaper.

- Cost optimization must not purchase unnecessary quantity merely to unlock an incentive.
- Voucher use is optional; saving a voucher is a valid AI decision.
- Loyalty rewards are optional; consuming a reward is not automatically optimal.
- The AI recommendation never equals financial authorization.
- User selection never bypasses policy.
- Policy denial cannot be overridden by the AI, frontend or user through a client-side parameter.
- The mandate cannot be modified by the AI during the transaction.
- Server-authoritative state is revalidated before Razorpay order creation.
- Any material financial action must be represented in the audit trail.

24. Functional Error Conditions

Condition	Required behavior
Ambiguous user goal	Ask for minimum necessary clarification; do not invent critical constraints.
No suitable products	Explain shortage and request a revised goal/constraint or stop safely.
Out of stock	Exclude/replan using current stock; never charge for unavailable inventory.
Price changed	Recalculate and revalidate before checkout.
Voucher expired	Remove voucher and recompute; do not claim savings.
Voucher not eligible	Do not apply; explain if relevant.
Loyalty reward unavailable	Do not apply; recompute.
Budget exceeded	Policy DENY; AI re-optimizes without changing mandate.
Category prohibited	Policy DENY; AI removes/replaces the item.
Per-item limit exceeded	Policy DENY; AI finds a compliant alternative.
Payment failure	Order remains unpaid/failed according to authoritative payment state; user can retry where supported.
Duplicate webhook	Ignore/reconcile idempotently without double-processing.

25. Minimum Acceptance Scenarios

- A1 — Natural-language goal produces structured requirements.
- A2 — Catalog search returns authoritative candidates.
- A3 — Cheaper poor-quality product is rejected in favor of an acceptable alternative.
- A4 — Pack combination satisfies quantity at lower/equal cost.
- A5 — Best Value and Best Quality differ when the candidate data supports a trade-off.
- A6 — AI recommends one basket and user can choose the other.
- A7 — 5% voucher on a ■100 basket can be marked SAVE when future value/validity supports saving.
- A8 — Materially valuable voucher can be marked USE.
- A9 — Loyalty reward can be evaluated and applied only when eligible.
- A10 — Basket is recalculated server-side after user selection.
- A11 — Policy denies a basket above the mandate.
- A12 — AI recovers from the denial without modifying the mandate.
- A13 — Authorized basket creates a Razorpay Test Mode order.
- A14 — Payment verification/webhook updates authoritative payment state.
- A15 — Audit trail contains enough information to reconstruct the transaction.

26. Functional Definition of Done

The MVP is functionally complete only when every mandatory requirement above is implemented, the minimum acceptance scenarios pass, the payment path is verified in Test Mode, and the canonical failure/recovery path is demonstrable.

This document must be read together with Document 01. If an implementation idea conflicts with the locked scope or authority model in Document 01, the locked master specification takes precedence.